

Multiple Linear Regression – Multi-Channel Marketing Analysis

Introduction

This project analyzes the relationship between multiple marketing channels and Sales using Multiple Linear Regression.

The analysis includes:

Exploratory Data Analysis (EDA) Categorical variable encoding Multicollinearity testing using VIF Regression modeling with statsmodels Residual diagnostics Business interpretation of coefficients

The objective is to determine which marketing channels significantly influence Sales and provide data-driven recommendations for marketing budget allocation.

1. Import Libraries

```
In [1]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

from sklearn.preprocessing import LabelEncoder

from scipy import stats
```

2. Load Dataset

```
In [2]: df = pd.read_csv("marketing_and_sales_data_evaluate_lr.csv")

df.head()
```

```
Out[2]:
```

	TV	Radio	Social_Media	Sales
0	16.0	6.566231	2.907983	54.732757
1	13.0	9.237765	2.409567	46.677897
2	41.0	15.886446	2.913410	150.177829
3	83.0	30.020028	6.922304	298.246340
4	15.0	8.437408	1.405998	56.594181

3. Initial Exploration

```
In [3]: print("Checking for information")
df.info()

print("\nChecking for missing Values")
df.isnull().sum()
```

```
Checking for information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4572 entries, 0 to 4571
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   TV              4562 non-null   float64
1   Radio           4568 non-null   float64
2   Social_Media    4566 non-null   float64
3   Sales           4566 non-null   float64
dtypes: float64(4)
memory usage: 143.0 KB
```

Checking for missing Values

```
Out[3]: TV              10
Radio              4
Social_Media       6
Sales              6
dtype: int64
```

```
In [4]: print("\nChecking for Statistics")
df.describe()
```

Checking for Statistics

Out[4]:

	TV	Radio	Social_Media	Sales
count	4562.000000	4568.000000	4566.000000	4566.000000
mean	54.066857	18.160356	3.323956	192.466602
std	26.125054	9.676958	2.212670	93.133092
min	10.000000	0.000684	0.000031	31.199409
25%	32.000000	10.525957	1.527849	112.322882
50%	53.000000	17.859513	3.055565	189.231172
75%	77.000000	25.649730	4.807558	272.507922
max	100.000000	48.871161	13.981662	364.079751

We begin by inspecting the dataset structure, checking for missing values, and understanding the distribution of numerical variables.

4. Encode Categorical Variables

Since TV and Influencer are categorical:

```
In [5]: le_tv = LabelEncoder()
le_influencer = LabelEncoder()

df['TV'] = le_tv.fit_transform(df['TV'])
df['Influencer'] = le_influencer.fit_transform(df['Influencer'])

df.head()
```

```

-----
KeyError                                Traceback (most recent call last)
File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3812, in Index.get_loc(self, key)
    3811 try:
-> 3812     return self._engine.get_loc(casted_key)
    3813 except KeyError as err:

File pandas\_libs\index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File pandas\_libs\index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

File pandas\_libs\hashtable_class_helper.pxi:7088, in pandas._libs.hashtable.PyObjectHashTable.get_item()

File pandas\_libs\hashtable_class_helper.pxi:7096, in pandas._libs.hashtable.PyObjectHashTable.get_item()

```

KeyError: 'Influencer'

The above exception was the direct cause of the following exception:

```

KeyError                                Traceback (most recent call last)
Cell In[5], line 5
      2 le_influencer = LabelEncoder()
      4 df['TV'] = le_tv.fit_transform(df['TV'])
----> 5 df['Influencer'] = le_influencer.fit_transform(df[      ])
      7 df.head()

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:4113, in DataFrame._getitem__(self, key)
    4111 if self.columns.nlevels > 1:
    4112     return self._getitem_multilevel(key)
-> 4113 indexer = self.columns.get_loc(key)
    4114 if is_integer(indexer):
    4115     indexer = [indexer]

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3819, in Index.get_loc(self, key)
    3814 if isinstance(casted_key, slice) or (
    3815     isinstance(casted_key, abc.Iterable)
    3816     and any(isinstance(x, slice) for x in casted_key)
    3817 ):
    3818     raise InvalidIndexError(key)
-> 3819     raise KeyError(key) from err
    3820 except TypeError:
    3821     # If we have a listlike key, _check_indexing_error will raise
    3822     # InvalidIndexError. Otherwise we fall through and re-raise
    3823     # the TypeError.
    3824     self._check_indexing_error(key)

```

KeyError: 'Influencer'

Categorical variables were encoded into numerical representations to allow regression modeling.

5. Correlation Matrix

```
In [ ]: plt.figure(figsize=(8,6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```

Interpretation:

Check relationships among predictors.

High correlations may indicate multicollinearity.

6. VIF (Variance Inflation Factor)

```
In [ ]: X = df.drop("Sales", axis=1)

vif_data = pd.DataFrame()
vif_data["Feature"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values, i)
                    for i in range(X.shape[1])]

vif_data
```

VIF values above 5 may indicate moderate multicollinearity, while values above 10 suggest severe multicollinearity.

7. Build Multiple Linear Regression Model

```
In [ ]: X = df.drop("Sales", axis=1)
y = df["Sales"]

X = sm.add_constant(X)

model = sm.OLS(y, X).fit()

print(model.summary())
```

8. Key Interpretation Section

Coefficient Interpretation

- Holding all other variables constant, a one-unit increase in Radio advertising spend is associated with an increase in predicted Sales equal to the corresponding regression coefficient.

- If the p-value of a variable is less than 0.05, the predictor is considered statistically significant.
- Adjusted R-squared explains the proportion of variation in Sales explained by the independent variables while accounting for model complexity.
- Variables with insignificant p-values may contribute little predictive power and can be considered for exclusion.

9. Residual Diagnostics

Residual Plot

```
In [ ]: predictions = model.predict(X)
residuals = y - predictions

plt.figure(figsize=(8,5))
sns.scatterplot(x=predictions, y=residuals)
plt.axhline(y=0, color='red', linestyle='--')
plt.xlabel("Predicted Values")
plt.ylabel("Residuals")
plt.title("Residual Plot")
plt.show()
```

Histogram of Residuals

```
In [ ]: plt.figure(figsize=(8,5))
sns.histplot(residuals, kde=True)
plt.title("Distribution of Residuals")
plt.show()
```

Q-Q Plot

```
In [ ]: sm.qqplot(residuals, line='45')
plt.title("Q-Q Plot")
plt.show()
```

10. Business Recommendation

Business Recommendation

Based on the regression analysis, marketing channels with statistically significant positive coefficients should receive increased budget allocation because they contribute positively to Sales performance.

The Adjusted R-squared value indicates that the model explains a substantial proportion of Sales variability, suggesting good predictive performance.

Residual diagnostics show whether regression assumptions such as linearity, normality, and homoscedasticity are reasonably satisfied.

Management should prioritize investment in channels with:

- Significant p-values (< 0.05)
- Positive coefficient impact
- Strong contribution to overall model performance

Channels with weak statistical significance may require reduced spending or further investigation.

In []: